SPHERE WEB AUTOMATION

Complete Competency Script Step-by-Step Walkthrough

How `tests/CompleteCompetency.spec.ts` climbs every level of one named competency — and the five independent guards it carries against the portal quietly handing back a different competency's assessment.

Script tests/CompleteCompetency.spec.ts · 1,739 lines · **1 test**, 37 helpers, **7 expect()**

Target /app/user/competency (login + COMPETENCY_NAME required)

Run COMPETENCY_NAME="…" npm run run:complete-competency

Date 12 September 2026

Contents

1	What this script does	Purpose, scope, entry points
2	The identity problem	Diagram — five guards against one bug
3	The climb	Diagram — one iteration, level by level
4	Configuration and inputs	Credentials forced at module load
5	Finding and opening a level	Three routes, and the card-tagging trick
6	Answering a quiz	Diagram — three key sources, one poison rule
7	Making a click stick	The radio subsystem, and why it exists
8	The Gemini fallback	Throttle, quota, model fallback
9	What green actually proves	Seven assertions and the Passbook cross-check
10	Artifacts, run variants, failure modes	Troubleshooting reference

How to read this document

Like the course-completion spec, this is a **driver** rather than an element sweep: one test that works through a real piece of the product until the portal says it is done. But it is the more interesting of the two to read, because it is shaped end to end by a single hazard — and Section 2 is about that hazard rather than about any function.

Sections 5 to 8 open up the machinery. Section 9 is the honest accounting, and it is unusually short: this script asserts more, and more precisely, than any other driver in the suite.

> **Line numbers in this document are live**
> Every reference points at `tests/CompleteCompetency.spec.ts` as it stands today — 1,739 lines, including the per-line explanatory comments carried across the suite.

> **Its sibling is `Competency.spec.ts`, and they are not the same thing**
> The file header is explicit: *"This is a SEPARATE script: it does not modify Competency.spec.ts."* The sibling splits the climb into one test per step for granular reporting; this one runs the whole climb as a single test against **one competency you name in the terminal**. Where they share logic — the poison rule, the Gemini plumbing, the question matcher — the shared parts live in `helpers/questionMatch.ts` or are deliberately duplicated so a change made for one cannot break the other.

1 What this script does

You give it the name of one competency. It clears every level of that competency, end to end, and then checks the portal's own Passbook to confirm the portal agrees.

A competency on Sphere is a ladder: up to five levels, each one an assessment, each locked until the level below it is cleared. Clearing all of them is how a learner demonstrates a skill, and it is the single most consequential workflow in the product — which is why there is a spec that does nothing else.

The shape of a run

1. **Force the credentials, before anything else.** Lines 29–44 run at module load, ahead of any fixture, so the auth harness resolves the account this flow intends rather than whatever `.env` happens to hold.
2. **Open the competency dashboard** and prove there is a session to drive — `assertLoggedIn()` throws a numbered, copy-pasteable fix rather than letting the run fail obscurely later.
3. **Climb.** For each level 1 to 5: open it, confirm it is the right competency, answer the quiz, read the verdict, return to the dashboard. Stop on the first level that does not clear, or when there is no further level — which is success, not failure.
4. **Check the Passbook.** The climb reports what the quizzes said; the Passbook is what the portal actually recorded. A run is only believable when the two agree.

Two inputs, and a hard rule about environments

Input	What happens without it
`COMPETENCY_NAME`	The test **throws immediately**, with the exact command to run. No default, and no silent skip.
`BASE_URL`	Defaults to production staging. But the localhost mirror **cannot do password login**, so pointing at it without CDP throws with three ways to fix it rather than failing at the login form.
`GEMINI_API_KEY`	Optional. Only used when a level's answer key is missing or poisoned; the run prints whether it is available before it starts.

> **In CDP mode it drives *your* tab, and may retarget the whole run**
> `beforeAll` does something unusual. Sphere keeps its session in per-tab `sessionStorage`, so a freshly opened tab lands on the public home page even in an authenticated browser. Rather than fight that, the script **reuses a tab you already have open**, preferring one on the environment you asked for and inside the app. If your only logged-in tab is on a different environment, it says so and **adopts that origin for the whole run** — because navigating your tab cross-origin would drop the session it needs. It also never closes a tab it did not open.

Entry points

Command	What it runs
`npm run run:complete-competency`	**The working entry point.** Headed, against staging, using whatever `COMPETENCY_NAME` is already set.
`USE_CDP=1 …`	Local development: attach to a Chrome you logged into by hand. Required on localhost, which cannot do password login.
`.\run-competency.ps1 "…"`	missing The wrapper the file header documents. It is **not in this repository** — see the note below.

> **The documented entry point does not exist**

The file header names `.\run-competency.ps1` as the easiest way to run the spec, and as the `-Local` form for CDP. It also appears inside both of the thrown error messages a user is most likely to read — the one for a missing competency name, and the one for a localhost `BASE_URL`. There is no such file anywhere in the repository.

So anyone who follows the header, or who forgets `COMPETENCY_NAME` and does what the error tells them, is sent to a command that cannot run. Either restore the wrapper, or point the header and both error strings at `npm run run:complete-competency` and the `USE_CDP=1` form. The second is a small change and would make the failure messages truthful.

```
# PowerShell — the manual form from the file header
$env:BASE_URL="https://sphere.aastrika.org"
$env:COMPETENCY_NAME="Care of the newborn"
npx playwright test tests/CompleteCompetency.spec.ts --headed
```

Forty minutes, one test

`test.setTimeout(40 * 60 * 1000)`. Five levels of quiz is slow on its own; when the answer key is poisoned and Gemini is answering, the self-throttle adds 6.5 seconds between questions. The ceiling is generous so a slow climb finishes rather than being cut off mid-ladder.

2 The identity problem

One bug shaped this entire file. Understanding it explains most of the code you would otherwise find over-engineered.

Ask the dashboard for the "neonatal" competency and it will, under the right conditions, hand back the assessment for "Normal delivery" instead. That alone would be a nuisance. What makes it dangerous is what happens next: **the quiz is answered from its own answer key**, so the wrong competency's assessment is answered correctly, passes, and reports success. Nothing downstream notices. The run goes green for a competency it never touched.

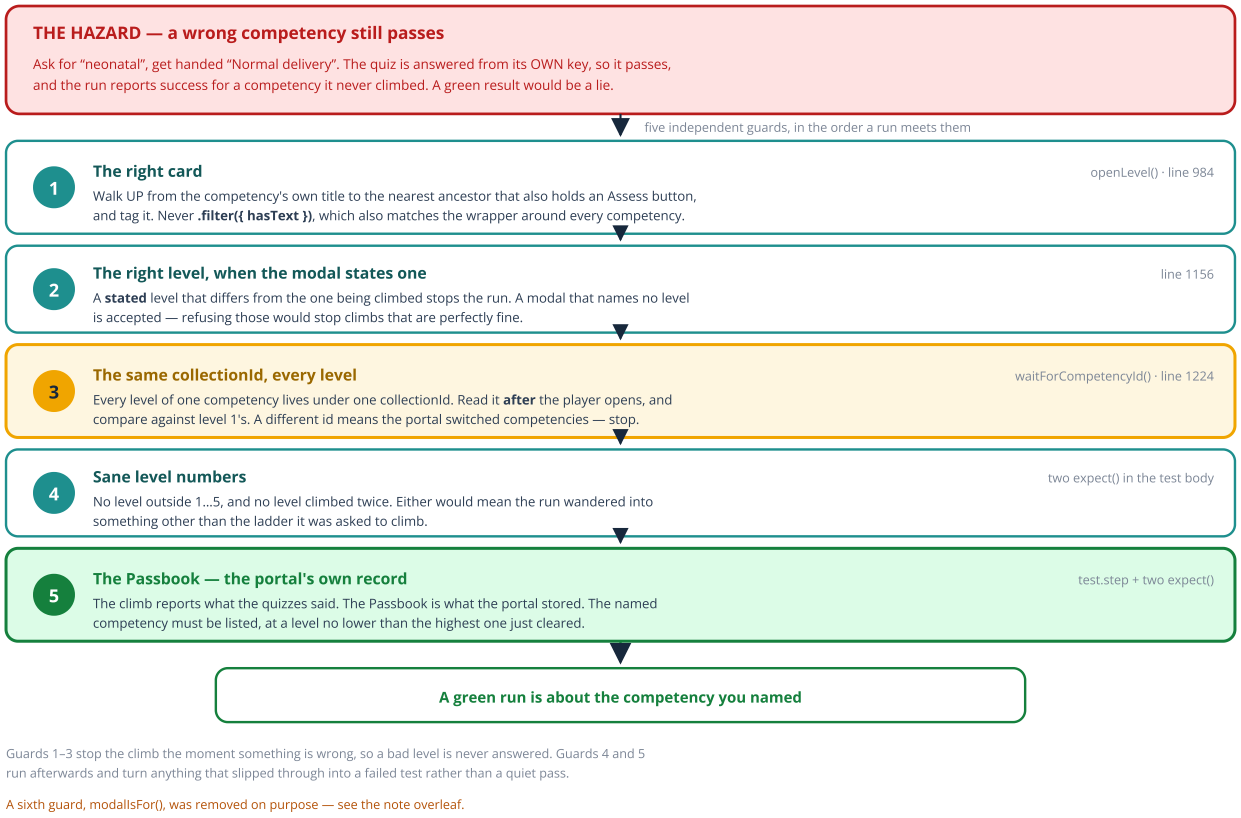


Figure 1 — five independent checks, all serving one purpose. The middle one is the load-bearing member; the last one is the only one that asks the portal rather than the screen.

2 The identity problem (continued)

The guard that was removed on purpose

There used to be a sixth check. `modalIsFor()` compared the competency name against the intro modal, and against the page behind it. It is still called — at lines 1141, 1273 and 1359 — and it still takes the name, but it now returns `true` unconditionally. The reasoning in its body is worth reading in full, because both halves of it failed for different reasons:

What it checked	Why it had to go
The modal's own heading	The modal shows the **level's** title — "Level 1 : Understands health of males and females..." — never the competency's name. Requiring the name there rejected every valid level. **Too strict to be usable.**
The page behind the modal	The dashboard lists *every* competency, so the name was always somewhere on the page. The check passed unconditionally and "confirmed" competencies it had never verified. **Too loose to mean anything.**

Neither version could work, so the identity question moved to something the screen cannot fudge: the `collectionId` in the player URL. The parameter is kept and marked with `void name` — the comment says why, "deliberately unused, not forgotten" — so the call sites did not have to change.

> **Its doc comment still describes the old behaviour**
> The JSDoc directly above `modalIsFor()` reads: *"Does the assessment now on screen belong to `name`? Checked on the modal first, then the page behind it..."* That is a description of the version that was deleted. The body immediately below it opens with "Deliberately NOT a name check" and returns `true`.
>
> A reader who trusts the doc comment — or a tool that surfaces it on hover — will believe a name check is happening when none is. The fix is to replace the JSDoc with the body's own explanation, which is already written and already correct. As it stands the two most authoritative-looking parts of the function contradict each other.

Why the id is read late

`waitForCompetencyId()` is called *after* the quiz player has opened, not off the intro modal — and that ordering was learned the hard way. The comment in the test body records it:

```
// Read it AFTER the player has actually opened. Reading it here, straight
// off the intro modal, gave an empty id at level 2 and the check skipped
// itself — which is how a Pregnancy Identification quiz still got through.
```

The intro modal appears **before** the app navigates to `/viewer/quiz/…`, so at that moment the URL carries no `collectionId` at all. An empty id is indistinguishable from "unknown", the guard treated it as nothing to check, and the wrong competency sailed past. So the helper now clicks Start Now itself — that click *is* what triggers the navigation — and then polls for up to 25 seconds for the id to appear.

What a guard does when it fires

None of the first three guards fails the test. Each one prints a ⛔ line explaining what it saw and `break`s out of the climb. The failure arrives afterwards, from the assertions in Section 9: a run that stopped at level 1 has fewer cleared levels than it should, or has none at all, and the expectations catch that.

That separation is deliberate and it is the right shape. The guards' job is to **stop doing damage** — do not answer a quiz you cannot identify — and the assertions' job is to **report**. Mixing them would mean a guard throwing from deep inside a helper, with a stack trace instead of the summary line that tells you which competency was opened instead.

3 The climb

One iteration per level, up to five. Three of the four ways out are failures; the fourth — running out of levels — is how a successful run ends.

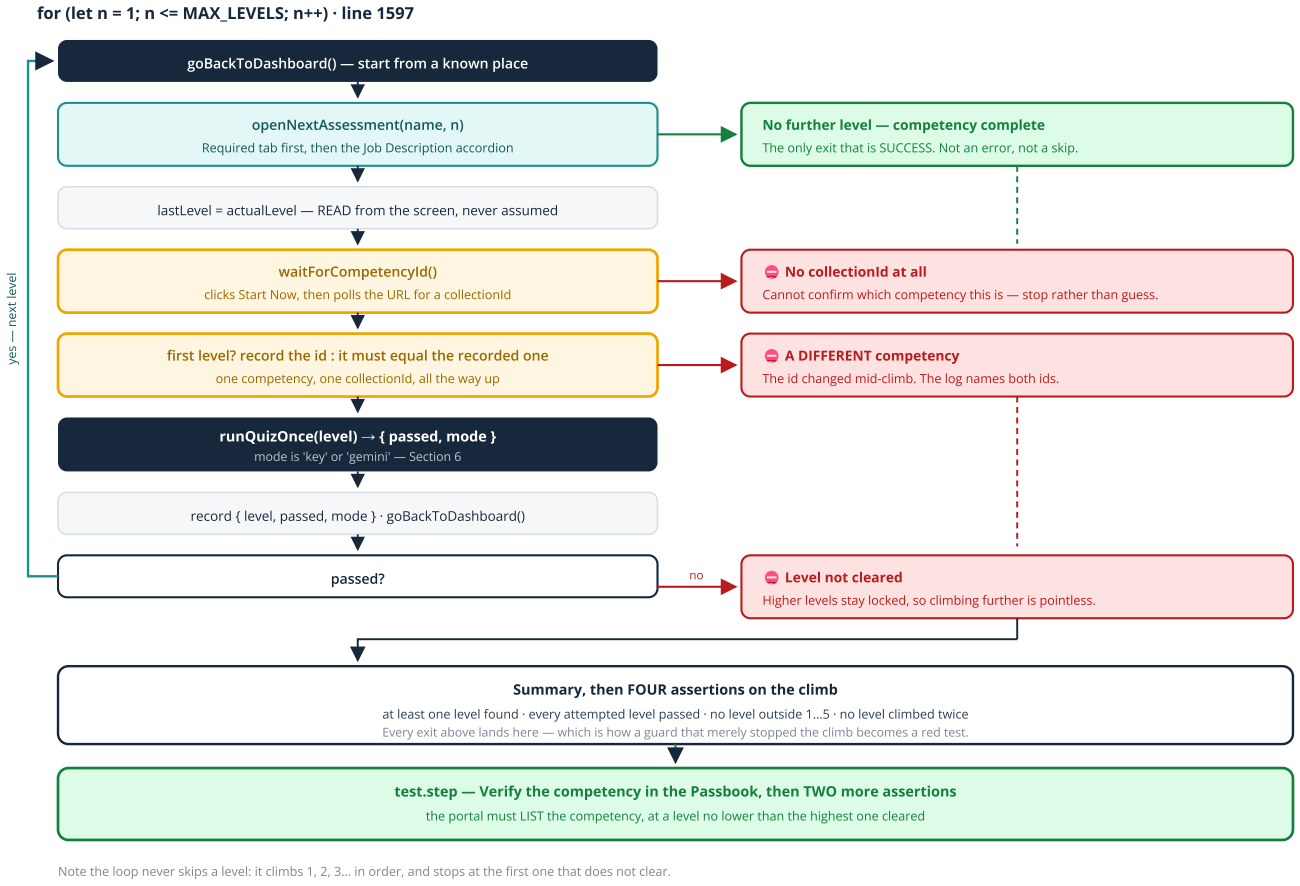


Figure 2 — the climb. Four ways out, one of them green. The band at the bottom is where every exit is finally judged.

4 Configuration and inputs

Most of this file's configuration is unremarkable. The first fifteen lines are not — they run before anything else in the process, and they are the reason the account in the log is the account you configured.

Credentials, resolved at module load

Lines 29–44 execute when the module is imported, ahead of every fixture. That timing is the whole point: `resolveCreds()` inside the auth harness reads these variables at fixture time, so setting them here decides which account the run authenticates as.

```
process.env.TEST_USER_EMAIL =
  process.env.COMPETENCY_USER_EMAIL || process.env.TEST_USER_EMAIL || 'sphereid041@yopmail.com';
```

Three levels of precedence, most specific first:

Source	When it wins
`COMPETENCY_USER_EMAIL / _PASSWORD`	An opt-in override for this flow only. Always wins.
`TEST_USER_EMAIL / _PASSWORD`	Whatever `.env` or the shell already set, if no flow-specific override exists.
`sphereid041@yopmail.com`	The standard learner, as a last resort.

> **The bug this shape fixed**
> The source records it in its own comment: the assignment **used to be unconditional**. Editing `.env` changed nothing, the run silently stayed on `sphereid041`, and — the part that actually hurts — **the account printed in the log was not the account you had configured**. A run could be investigated for an hour against the wrong learner's data. The `||` chain is a small change that made the log trustworthy.

Environment: one hard rule

This flow logs in with a password, and the localhost mirror does not support password login. So two things happen. First, `BASE_URL` is defaulted to staging at line 46 rather than left unset — the comment says *"don't silently auth-fail — make it obvious."* Second, the test body refuses a localhost target outright, **unless CDP mode is on**:

```
if (/localhost|127\.0\.0\.1|:3000/.test(BASE_URL) && process.env.USE_CDP !== '1') throw …
```

The exception matters: attaching to a Chrome you logged into by hand works perfectly well on localhost, because no password is ever typed. The thrown error spells out all three routes rather than stating the problem and leaving you to work it out.

Constants

Constant	Value	What it governs
`MAX_LEVELS` line 85	**5**	The height of the ladder, and the upper bound the assertions check against. A level number outside 1…5 is treated as evidence the run wandered.
`GEMINI_RPM_DELAY` line 94	**6.5 s**	Minimum spacing between AI calls, sized for the free tier's per-minute limit.
`GEMINI_PRIMARY / _FALLBACK` lines 88, 90	`2.5-flash` `2.5-flash-lite`	The two models, with separate quotas — Section 8.

Constant	Value	What it governs
`COMPETENCY_URL` line 68	`/app/user/competency`	Declared with `let`, not `const`, because `setBaseUrl()` can rewrite it mid-run when the script adopts a CDP tab's origin.

Why `BASE_URL` is mutable here and nowhere else

Every other spec treats `BASE_URL` as a constant. This one declares it `let` (line 62) and provides `setBaseUrl()` (line 71) to change it along with every URL derived from it. That exists for exactly one situation: you are running in CDP mode, your only logged-in tab is on a different environment, and the choice is between adopting that environment or destroying your session by navigating away from it. The script adopts, announces what it did, and carries on.

5 Finding and opening a level

Three routes, tried in order, because the portal presents competencies differently depending on build and on whether the learner has started one.

Route	Line	How it works
Required tab `openLevel`	984	The short route. Find the competency's card on the Required tab and click its **Assess** (fresh) or **Re-Assess** (already started) button.
Competency TOC `openLevelFromToc`	1266	On some environments Assess does not open a modal — it navigates to a table of contents listing every level as a link. This picks the right level from that list.
Job Description `openViaJobDescription`	1310	The fallback when the Required tab has no card: competencies also live under Job Description, grouped by role in expansion panels. Slower, but it works when the Required tab is empty.

`openNextAssessment()` (line 1382) is the two-line chooser: try the Required tab, and fall through to Job Description if it did not work.

The card-tagging trick

This is the most interesting twenty lines in the file, and it is guard 1 from Section 2. The problem is finding the button that belongs to *this* competency, and both obvious approaches fail:

Too high

- `.filter({ hasText: name })` matches every ancestor that contains the name
- ...including the wrapper around **all** competencies
- `.first()` then takes that wrapper
- The button clicked belongs to whichever competency comes first

Too low

- Going to the nearest card ancestor of the title...
- ...lands on the title's own little wrapper
- which has no button in it at all
- so nothing is clickable

The right element is **the closest ancestor containing both the title and an Assess button**, and there is no CSS selector for that. So the script goes into the browser and finds it by walking:

1. Clear any `data-pw-card` attribute left by a previous call — the tag is reused every iteration.
2. Find elements whose **own** normalised text *equals* the competency name. `===`, not `includes`: a container that merely mentions the name is not the title.
3. From each such title, climb up to 12 levels, checking at each step whether that ancestor holds a button reading exactly `Assess` or `Re-Assess`.
4. Tag the first ancestor that does with `data-pw-card="1"`, and return.
5. Back in Node, select `[data-pw-card]` — now a single, unambiguous element — and click its button.

The Re-Assess button is looked for first; if it is absent the competency has never been started, so the script falls back to `/^assess$/i` — anchored, so it cannot match "Re-Assess" itself.

> **Marking the DOM to make it selectable**
>
> Setting an attribute from `page.evaluate()` purely so Playwright can select it afterwards is an unusual move, and worth recognising as a legitimate one. It is the answer whenever the element you want is defined by a *relationship* — nearest ancestor of A that also contains B — rather than by any property a selector can express. The alternative is a long, brittle chain of `xpath=ancestor::` steps that encodes the current markup depth.

6 Answering a quiz

Competency quizzes are the ones the platform deliberately protects. The copy the browser receives has its answers stripped — so the script goes and gets the original.

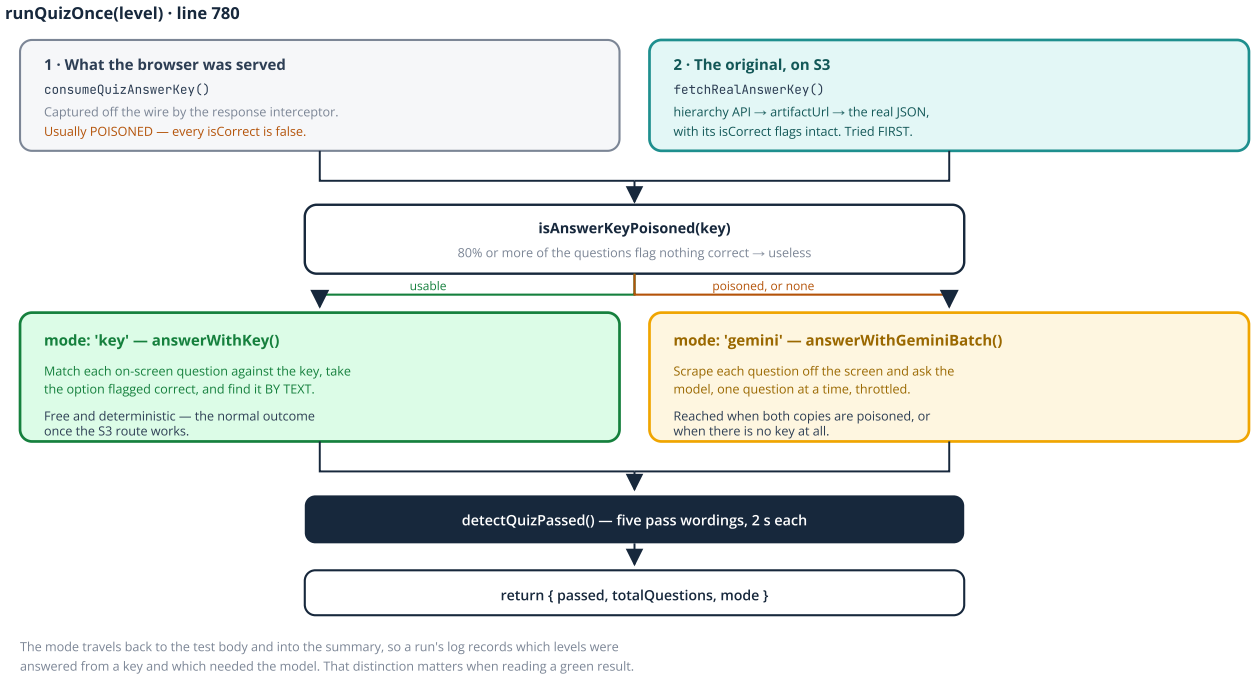


Figure 3 — two key sources, one poison rule, two answering modes. The S3 route is tried first and is what keeps the AI out of most runs.

6 Answering a quiz (continued)

Getting the original key

`fetchRealAnswerKey()` (line 213) is the part worth understanding. It reconstructs the path from "a quiz is open in the viewer" to "the published JSON on S3", using nothing but the URL and the app's own headers:

1. **Two ids out of the viewer URL.** `/viewer/quiz/(do_…)` gives the quiz; `collectionId=(do_…)` gives the course it belongs to. Missing either means this path cannot work, and it returns `null` immediately.
2. **Steal the headers.** `setupApiHeaderCapture()` (line 200) watches the app's outgoing requests and copies the `org`, `rootorg` and `hostpath` values off the first proxy call it sees. The comment is right that this is "far more reliable than hard-coding them, since they vary by environment".
3. **Build `hostPath` as a flat token.** The API wants `sphere.aastrika.org` written as `sphere_aastrika_org` — scheme stripped, `:` and `/` replaced, trailing underscores trimmed. Used only as a fallback when no header was captured.
4. **Ask for the hierarchy** with `hierarchyType=detail` — that parameter is what makes the response carry `artifactUrl` at all — using the **page's own request context**, so the session cookies ride along.
5. **Walk the course tree** depth-first to find the node whose `identifier` is this quiz, and take its `artifactUrl`.
6. **Download the artifact** and accept it only if it actually contains questions.

> **An explicit stack instead of recursion**
> The tree walk uses `const stack = [root]` and a `while (stack.length)` loop rather than a recursive function, and the comment says why: the course tree can be deep and this must not blow the call stack. It is a small thing, but it is the kind of small thing that turns an occasional mysterious crash into no crash at all.

The poison rule

Identical to the course-completion spec's, and deliberately so — the comment points at the shared origin. A question is poisoned when **every** one of its options is flagged `isCorrect: false`; a key is poisoned when 80% or more of its questions are. The threshold rather than "all" means a handful of genuinely-unflagged questions does not throw away a usable key, and an empty key counts as poisoned so the caller has one condition to check rather than two.

The interception itself is deliberately loose about *where* the JSON came from:

```
// Match the quiz JSON on ANY host (prod serves it from sunbirdcontent.s3,
// but the localhost mirror may proxy it from a different origin). We still
// validate the payload shape below, so a loose URL match is safe.
if (!/_quiz\.json/i.test(url) && !/\/quiz\/.*\.json/i.test(url)) return;
```

Shape validation — a non-empty `questions` array — is what makes the loose URL match safe. The captured key is then **consumed**: `consumeQuizAnswerKey()` takes it and clears the slot, so level 1's key cannot silently be reused on level 2.

Reading the verdict

`detectQuizPassed()` (line 949) tries five wordings — *passed the assessment*, *congratulations*, *you have passed*, *level cleared*, *assessment cleared* — each with a two-second window, and treats "none of them found" as a fail.

> **A missing verdict reads as a failure**
> That default is the safe direction for this script — a level is only recorded as cleared when the portal actually said so — but it is worth naming, because it means **a rendering change to the results screen shows up as**

a failed level, not as a broken test. If a run reports a level failed and the log shows a full set of correct answers, the wording list is the first thing to check.

7 Making a click stick

Ninety lines to select a radio button. That sounds absurd until you read what each of them is for — and what an unanswered question costs.

The stakes are stated at the top of `selectRadioOption()` (line 676): *"An unanswered question quietly costs a mark, which is how a level fails with a perfectly good answer key."* The script can know the right answer, click it, and still fail the level — because the click did not register and nothing said so.

Three bugs are encoded here

The bug	The fix, and why it is where it is
Clicking a hidden group	The quiz keeps earlier questions' radio groups **mounted but hidden**. So `mat-radio-group` with `.first()` aimed every click at a stale, off-screen group and nothing ever registered. The fix is `mat-radio-group:visible` — and, critically, `radioIsChecked()` resolves the group the same way, so the clicker and the verifier always mean the same element.
Synthetic events	Angular Material acts on *trusted* events. The original version only ever fired synthetic ones from inside `page.evaluate()`, which Material ignored. A real Playwright click now comes first; the DOM-level manipulation is the last resort, not the first move.
Reading the state too early	The old code re-read the checked state in the same synchronous pass, **before Angular's change detection had run** — so a click that had worked looked failed, the next strategy fired, and the two could cancel each other out. Every strategy now waits 350 ms before reading back.

Four strategies, three passes

Each pass escalates, and each step verifies before moving on:

1. **Click the label.** What a person does. Three wrapper shapes are tried — `label`, `.mdc-form-field`, `.mat-mdc-radio-touch-target` — and whichever exists gets the click.
2. **Click the component itself.** `force` is applied only from the second pass onward: try it honestly first, and only overrule the actionability checks once the honest attempt has failed.
3. **Check the native input.** `force: true` here always, because Material hides the real `<input>` behind a styled wrapper that would otherwise fail the visibility check.
4. **Drive it from the DOM, and tell Angular.** Click the input; if it is still not checked, set `checked = true` by hand and dispatch `input` and `change` events so the Angular form control notices. Without those two events the DOM and the framework disagree, and the framework is the one that counts.

How "checked" is measured

`radioIsChecked()` asks three different questions and accepts any of them, because Material's state lives in three places and different builds update different ones:

```
return target.classList.contains('mat-mdc-radio-checked')
  || target.getAttribute('aria-checked') === 'true'
  || !!target.querySelector('input[type="radio"]:checked');
```

> **It says "UNANSWERED" out loud**
> After twelve failed attempts the function logs `⚠️ this question is UNANSWERED` and returns `false`. That wording is chosen carefully: the caller falls back to option A, so the run continues and may still pass — but the log now contains the one word that explains a level failing with a correct answer key. Without it the failure would be invisible.

Intermediate retries are logged too (`↻ Option N did not take — retry 2/3`), so the log shows how hard the script had to fight. A level that cleared after several of those is worth a look even though it passed.

8 The Gemini fallback

Reached only when both copies of the key are poisoned. Smaller and simpler than the course-completion spec's version, because the shape of the problem is different.

A competency quiz has five to ten questions and there are at most five levels, so the total volume is modest. That is why this file asks **one question at a time** (`askGeminiOne`, line 410) rather than batching a whole assessment into a single prompt: per-question prompts are more accurate, and the quota can absorb them here.

Piece	Line	What it does
`throttle()`	322	Sleeps so no two calls are closer than `GEMINI_RPM_DELAY` (6.5 s), sized for the free tier's per-minute limit. Skipped on the first call.
`callGemini()`	334	One REST call, returning a result object rather than throwing — a network failure here is an ordinary outcome.
`parseRetryDelayMs()`	514	Digs the server's suggested wait out of a rate-limit body, so the script waits exactly as long as it is told to rather than guessing.
`parseAnswerMap()`	368	Turns the model's reply into usable indices. Models are inconsistent about formatting, so this has to be forgiving.
`askGeminiOne()`	410	One question, one answer. Handles the model fallback from `2.5-flash` to `2.5-flash-lite`.
`geminiQuotaExhausted`	319	A module-level flag. Once the per-day quota is gone, later requests are skipped outright instead of being retried into the same wall.

Two models means two daily budgets
`gemini-2.5-flash` and `gemini-2.5-flash-lite` have separate free-tier quotas, so falling back to the lighter model is not only about staying available under a per-minute limit — it roughly doubles what a day's runs can consume before the AI path stops working altogether.

What it means for a result

The mode of each level is recorded and printed in the summary:

```
"Care of the newborn" — levels cleared this run: 3
[{"level":1,"passed":true,"mode":"key"},{"level":2,"passed":true,"mode":"key"},
 {"level":3,"passed":true,"mode":"gemini"}]
```

A level cleared via `key` is a deterministic result: the same run would pass again. A level cleared via `gemini` depended on a third-party model answering clinical multiple-choice questions correctly, and might not repeat. Both count as passes, and the summary is the only place that distinction survives — which is a good reason to read it rather than just the result line.

9 What green actually proves

Seven assertions, six of them unconditional. For a driver script that is unusually strong — and the last two are the ones that make a green run believable.

#	What it checks	The assertion	Why it is there
1	**The dashboard rendered**	`expect(getByText(/competency dashboard/i).first()).toBeVisible({ timeout: 30_000 })`	assert A 30-second window, because the dashboard is slow to paint. Fails fast if the run never got as far as the page it needs.
2	**A level was found**	`expect(levels.length).toBeGreaterThan(0)`	assert Zero levels means the competency was never located at all — a different failure from "a level did not pass", and worth separating.
3	**Every attempted level passed**	`expect(levels.every((l) ⇒ l.passed)).toBe(true)`	assert The core claim. Note it covers every level *attempted*, not a fixed count — a competency with three levels passes on three.
4	**Level numbers are sane**	`expect(bad).toHaveLength(0) where bad = levels outside 1…5`	assert Guard 4. A level number off the ladder means the run wandered somewhere else. The message lists the offenders.
5	**No level twice**	`expect(new Set(cleared).size).toBe(cleared.length)`	assert A `Set` drops duplicates, so a size mismatch means the same level was climbed twice — the other signature of a wandering run.
6	**The Passbook lists it**	`expect(passbook.found).toBe(true)`	assert **The one that matters most.** Everything above is the script reporting on itself; this asks the portal. The message says it plainly: "the portal did not record this climb".
7	**At the right level**	`expect(passbook.level).toBeGreaterThanOrEqual(highest)`	conditional Runs only when the Passbook entry actually states a level. `GreaterThanOrEqual`, not equality, because a previous run may legitimately have climbed higher.

The Passbook check

It is wrapped in `test.step()`, so the HTML report shows it as its own collapsible section rather than as a tail of console noise. Inside, it looks for a row naming the competency across four container shapes — `mat-card`, anything with a *card* class, a `tr`, anything with a *row* class — with a 20-second window because the Passbook loads slowly.

And when it does not find one, it does the thing that makes a failure diagnosable instead of mysterious:

```
🔍 Passbook entries seen:
   [0] Normal delivery — Level 2
   [1] Pregnancy identification — Level 1
   …
```

Ten entries, printed verbatim. If the portal recorded a *different* competency — the exact hazard from Section 2 — that list is where you see it.

Green means this

- The named competency was found and opened

Green does *not* mean this

- Every level attempted was passed
- All levels belonged to **one** competency, by collectionId
- Levels were 1…5, each climbed once
- **The portal recorded the climb in the Passbook**
- At a level no lower than the highest cleared

- That every level of the competency was climbed — running out is success, so a portal that stops offering levels early passes
- That answers were right for the right reasons, where `mode` was `gemini`
- That no question was left unanswered — those fall back to option A and can still pass
- That the Passbook level is exactly right, only that it is not lower
- That the same run would pass again on an AI-answered level

"No further level" is indistinguishable from "the portal stopped offering levels"
When `openNextAssessment()` returns `ok: false`, the script logs `🏁 competency complete` and breaks — treating it as success. That is right in the normal case: a three-level competency has nothing at level 4. But it is the same signal a portal gives when a level exists and simply cannot be opened, and assertion 2 only requires *one* level to have been found.

In practice assertion 7 catches most of it — the Passbook level would be lower than expected — but only when the entry states a level. If this run is a release gate, recording the expected level count per competency and asserting against it would close the gap.

Against the rest of the suite

Suite	Tests	Assertions	What green is evidence of
Complete Competency this one	1	**6 + 1**	One competency was genuinely climbed, and the portal agrees. The only driver in the suite that cross-checks its own result against stored state.
Course Completion	1	1	A course was completed end to end. Broader, but self-reported.
Course TOC	21	15 + 3	Every control on one page renders and responds.
My Courses	49	17 + 1	The learner's home page renders whatever the account has.

The Passbook check is the difference. Every other driver in this suite finishes by believing its own log; this one goes and asks the system of record, and fails if the two disagree.

10 Artifacts, run variants, failure modes

Reading the console

There is no bespoke report for this spec — the standard reporters run, and the console is where the detail lives.

Mark	Line	What it tells you
👤	Account: sphereid041@yopmail.com	`Which account the run will use. In CDP mode it says instead that the account is whoever is signed into that tab — because no login is performed.`
♻️	Reusing your tab: https://.../page/home	`CDP mode adopted an existing tab. If the origin differs from BASE_URL, the next lines say the run retargeted itself.`
🎯	Attempting level 2 of "Care of the newborn"	`One per iteration. The fastest way to see how far a run got.`
🔓	REAL answer key from S3: 8 questions	`The hierarchy route worked and bypassed the poisoned copy. The best line in the log.`
🆔	Competency id: do_1141...	`Recorded at level 1; every later level is compared against it.`
⛔	Level 2 belongs to a DIFFERENT competency	`A guard fired. The line names both ids, so you can see what was opened instead.`
⚠️	this question is UNANSWERED	`A radio click never registered after twelve attempts. Explains a level failing despite a good key.`
📗	Passbook: "Care of the newborn Level 3"	`What the portal actually recorded.`

Failure modes

Symptom	Cause	Fix
`No competency name provided`	`COMPETENCY_NAME` is unset.	Set it. Note the error text points at `run-competency.ps1`, which is not in this repository — use `npm run run:complete-competency` instead.
`BASE_URL is "http://localhost:3000"…`	The localhost mirror cannot do password login.	Either target a real environment, or set `USE_CDP=1` and attach to a Chrome you logged into by hand.
`Not signed in on …`	CDP mode attached to a tab with no session. Thrown by `assertLoggedIn()` *before* the run hunts for cards.	Follow the three numbered steps in the error. The check exists because an unauthenticated dashboard shows a shell with a spinner, which otherwise looks exactly like "competency not found".
`Assess / Re-Assess button not visible`	The card-tagging walk found the title but no ancestor holding a button, or the competency is not on the Required tab.	The run then tries the Job Description route automatically. If both fail, check the exact competency title — the title match is **exact** after normalisation, not a substring.
`⛔ no collectionId in …`	The player never navigated, so the URL carries no id.	Usually the intro modal's Start Now did not take. Watch the run headed; the 25-second poll is generous, so a timeout here means the click, not the wait.
`⛔ Level N belongs to a DIFFERENT competency`	Exactly the hazard this file exists to catch.	A real product defect, and the most valuable failure this spec produces. The log names both ids — file it with those.

Symptom	Cause	Fix
A level fails with a full set of correct answers	Either a radio click never registered, or the results screen changed wording.	Grep the log for `UNANSWERED`. If there is none, the wording list in `detectQuizPassed()` is the next thing to check.
Every level uses `mode: gemini`	The S3 route is failing — usually the captured API headers, or a changed hierarchy response shape.	Check for the 🔓 line. Without it, `fetchRealAnswerKey()` returned null and every level is depending on the model.
The Passbook assertion fails after a clean climb	The portal did not record what the quizzes reported.	The printed list of Passbook entries is the evidence. This is a genuine product finding, not a test problem.
The run hits the 40-minute timeout	Usually the Gemini path on several levels, at 6.5 s per question plus rate-limit waits.	Fix the key route rather than raising the timeout — see the row above about `mode: gemini`.

Where to start when a run goes red

Read the summary line first — `levels cleared this run` plus the JSON array. It tells you how far the climb got and how each level was answered, and that is usually enough to know whether you are looking at a product defect, a stale selector, or a quota problem.